```
# -*- coding: utf-8 -*-
"""Untitled0.ipynb

Automatically generated by Colab.

Original file is located at
    https://colab.research.google.com/drive/1S6CFKph7hInZbts_MQHN8VKuoOiXHxvQ
"""

# Commented out IPython magic to ensure Python compatibility.
# %pip install pandas numpy matplotlib seaborn openpyxl scikit-learn

#Importing required libraries for clustering

import pandas as pd                         #For data handlling
import numpy as np                          #For numerical operations
import matplotlib.pyplot as plt             #For visualization
import seaborn as sns                       #For better looking visualization

#Providing the data set file format

data = pd.read_csv(r"/content/solarpowergeneration dataset.csv")
data

# Basic dataset info

data.head()

data.tail()

data.dtypes

data.shape          #For checking rows and columns of the dataset

data.info           # Info about the dataset

data.describe()          #Summary statistics

data.describe()          #Summary statistics  #For checking missing values

data.isnull().values.any()    #For checking missing values are there are not

sns.heatmap(data.isnull(), cbar=False, cmap='viridis') # Visualizing

# Filling missing values

for col in data.columns:
    if data[col].dtype=='object':
        #categorical colums should be filled with mode
        data[col]=daa[col].fillna(data[col].mode()[0])
    else:
        #Numerical column should be filled with mean
        data[col]=data[col].fillna(data[col].mean())

# Rechecking missing values

print("After filling missing values:")
print(data.isnull().sum())

sns.heatmap(data.isnull(), cbar=False, cmap='viridis') #For visualizing mising values

# One-hot encoding

encoding=pd.get_dummies(data, drop_first=True)
print("Data types after encoding")
print(encoding)
print(data.dtypes)

from sklearn.preprocessing import LabelEncoder
# Create a LabelEncoder object
le = LabelEncoder()
#Apply LabelEncoder to all categorical (object) columns
for col in data.select_dtypes(include='object').columns:
 data[col] = le.fit_transform(data[col])
print("\nData types after encoding:\n")
print(data.dtypes)
data.hist(bins=15, figsize=(15, 10))
```

```
plt.suptitle('Numerical Variables')
plt.show()

# Pairplot

sns.pairplot(data)
plt.show()

# outlier detection boxplot

plt.figure(figsize=(10, 6))
sns.boxplot(data=data.select_dtypes(include='number'))
plt.title("Boxplots for All Numeric Columns")
plt.xticks(rotation=45)
plt.show()

# Scatter plot

sns.scatterplot(data)
plt.title('Scatter Plot (colored by Pclass)')
plt.xlabel('clustering x')
plt.ylabel('clustering y')
plt.show()

# Heat Map

numeric_dataset = data.select_dtypes(include=['number'])
correlation_matrix = numeric_dataset.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()

from sklearn.preprocessing import MinMaxScaler
# Apply Min-Max Normalization
scaler = MinMaxScaler()
normalized_data = scaler.fit_transform(data)
print (normalized_data)

from sklearn.preprocessing import StandardScaler
# Apply Standardization
scaler = StandardScaler()
standardized_data = scaler.fit_transform(data)
print(standardized_data)

from sklearn.model_selection import train_test_split
# Dependent variable y (column 7 → 'GDP')
y = data.iloc[:, 9]
# Independent variables X (all except column 7)
X = data.drop(data.columns[9], axis=1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
print(X_train, X_test, y_train, y_test)

print("X_train:\n", X_train.head())
print("X_test:\n", X_test.head())
print("y_train:\n", y_train.head())
print("y_test:\n", y_test.head())

# Print shapes
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

# Fitting the model to the training data
from sklearn.linear_model import LinearRegression
regression =  LinearRegression()
regression.fit(X_train , y_train)

y_pred = regression.predict(X_test)
y_pred

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np
mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
```

```
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("R² Score:", r2)

plt.figure(figsize=(6, 4))
plt.scatter(y_test, y_pred, alpha=0.6, color='blue')
plt.xlabel("Actual Power Generated")
plt.ylabel("Predicted Power Generated")
plt.title("Linear Regression - Actual vs Predicted")
plt.show()

from sklearn.tree import DecisionTreeRegressor
model = DecisionTreeRegressor(random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print(y_pred)

plt.figure(figsize=(6, 4))
plt.scatter(y_test, y_pred, alpha=0.6, color='blue')
plt.xlabel("Actual Power Generated")
plt.ylabel("Predicted Power Generated")
plt.title("Decision Tree Regression - Actual vs Predicted")
plt.show()

# Random Forest Regression
from sklearn.ensemble import RandomForestRegressor
model = RandomForestRegressor(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# Predict on test set
y_pred = model.predict(X_test)
y_pred

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("Model Evaluation Metrics:")
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("R² Score:", r2)

plt.figure(figsize=(6, 4))
plt.scatter(y_test, y_pred, alpha=0.6, color='blue')
plt.xlabel("Actual Power Generated")
plt.ylabel("Predicted Power Generated")
plt.title("Random Forest Regression - Actual vs Predicted")
plt.show()

from sklearn.neighbors import KNeighborsRegressor
model = KNeighborsRegressor(n_neighbors=5)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print(y_pred)

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("Model Evaluation Metrics:")
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("R² Score:", r2)

plt.figure(figsize=(6, 4))
plt.scatter(y_test, y_pred, alpha=0.6, color='blue')
plt.xlabel("Actual Power Generated")
plt.ylabel("Predicted Power Generated")
plt.title("KNN Regression - Actual vs Predicted")
plt.show()
```

```
# Support Vector Machine (SVM) Regression
from sklearn.svm import SVR
model = SVR(kernel='rbf', C=100, gamma=0.1, epsilon=0.1)
model.fit(X_train, y_train)

y_pred_scaled = model.predict(X_test)
y_pred

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
print("Model Evaluation Metrics:")
print("Mean Absolute Error (MAE):", mae)
print("Mean Squared Error (MSE):", mse)
print("Root Mean Squared Error (RMSE):", rmse)
print("R² Score:", r2)

plt.figure(figsize=(6, 4))
plt.scatter(y_test, y_pred, alpha=0.6, color='blue')
plt.xlabel("Actual Power Generated")
plt.ylabel("Predicted Power Generated")
plt.title("SVM Regression - Actual vs Predicted")
plt.show()
```